

Chapter 11

Contents

Chapters

- 11. Interfacing Displays & Keyboards
 - 11.1 Introduction to Displays
 - 11.2 Interfacing Seven Segment Display
 - 11.3 Interfacing LCD Display
 - 11.4 Interfacing Keypad

11.1 Introduction to Displays

A microcontroller-based system is typically designed for acquiring data from sensors. The data is then processed and displayed on any one of the display types. A list of display types is presented here.

- Segment Display
- LCD Display
 - CCFL backlit LCD
 - LED backlit LCD
 - TFT LCD
 - IPS LCD
 - Mini-LED & Micro-LED
 - QLED
- OLED
 - OLED
 - QDLED
 - AMOLED
 - Super AMOLED
 - Dynamic AMOLED, Super Retina XDR

Segment Displays

Segment displays are composed of different segments that could be switched ON or OFF to display digits or alphanumeric characters. An LED or a Liquid crystal may be used for each segment. The Segment displays do not offer the flexibility to display all the types of characters or symbols. Hence LCDs and OLEDs are preferred. A 7-Segment display is the commonly used segment display.

LCD – Liquid Crystal Display

LCD displays consist of thin layer of crystal solution that is sandwiched between two polarized glass panels. Additionally, it uses a Cold Cathode Fluorescent lamp (CCFL) or Light Emitting Diode (LED) backlight and a set of million transistors that supply power to the crystals. The transistors act as switches to open & close based on the supplied power and functions as filter to control the passage of light through the polarized layer to create the required image. The passage of light is controlled per pixel. A traditional monochromatic filter uses a single transistor. A coloured LCD uses three transistors, one each for Red, Green & Blue - RGB colour filter. The combination of RGB could create any required colour.

Traditional LCDs uses CCFL as the backlight source and the later variations use LEDs backlight and hence are called LED displays. i.e. LED displays are variations of LCDs that consume very little power. The disadvantage of the first-generation LCDs is they used

Twisted Nematic - TN technique that creates colour distortion with increase in viewing angles i.e. when viewed from the side. Other techniques used are Super twisted Nematic-STN, Film compensated STN - FSTN, Double Layer STN – DSTN, ESTN and ISTN etc.

TFT LCD – Thin Film Transistor Display

TFT LCD is an advanced version of LCD that makes use of active matrix addressing scheme, i.e. each pixel is individually connected by a transistor and a capacitor (individual pixel could be addressed). The thin layer of transistors is deposited on the back of the screen. Traditional LCDs doesn't have capacitor and is not addressed individually (Passive).

The advantage of TFT LCDs is that they have higher contrast, lower production cost, Sharper, faster and brighter. The disadvantage is that they demand higher energy, less viewing angles with limited colour reproducibility.

IPS LCD – In-Plane Switching Display

The colour distortion of first-generation LCDs is overcome in IPS LCD's by aligning the liquid crystals with the display. Additionally, IPS LCDs also have the capability for superior colour reproduction.

PLS LCD – Plane to Line Switching Display

PLS LCD is the technology developed by Samsung that has similar characteristics of IPS LCD. It has lesser contrast levels w.r.t OLEDs.

Mini-LED & Micro-LED Display

Mini-LED is a technique based on miniaturization of LEDs (typically 0.008in) in an LCD display. Miniaturized LEDs allow for incorporating more LEDs in the backlight which increases the contrast levels and higher peak brightness of the display. Micro-LEDs (typically 0.002in) are further miniaturized versions of mini-LEDs that are currently expensive for uses other than Smartphone's and TVs.

QLED – Quantum dot LED display

QLEDs that are currently present in TVs make use of a quantum dot filter. The filter is a film of tiny molecules that emits their own-coloured light when it is hit with a backlit LED light (Ex: Mini-LEDs). Quantum dots emit their own light, efficient and produce brighter image and deeper colours.

OLEDs – Organic Light Emitting Diode (or) Organic Electroluminescent Diode

OLEDs make use of organic material (molecules or polymers) (do not have polarizing layers, crystals like LCDs) that emits light when a current is passed through it. It consists of thin sheets of electroluminescent material that produces their own light and hence OLEDs does not have any backlit light whereas LCD displays make use of backlit LEDs.

The Individual pixels in OLEDs are always OFF and current need to be passed to only those that should emit the light. Each sub pixel contains different molecule, react to applied voltage and produces different colours.

Since black pixels are OFF the energy requirement is minimized, and it is further less when blacker or darker colours need to be displayed. OLEDs have better contrast ratios and very fast refresh rate w.r.t LCDs as black pixels are off. Since a backlit is not required OLEDs could be made thinner and flexible.

However, the energy requirement is higher for displaying lighter colours w.r.t LCD's. Also, they are not as visible in daylight compared to LCDs. Additionally, screen burn-in, degradation (organic properties) affects the performance.

PMOLED – Passive Matrix Organic Light Emitting Diode

It is a specific type of thin film display with passive matrix addressing scheme.

AMOLED – Active-Matrix Organic Light Emitting Diode

It is a specific type of thin film display with active matrix addressing scheme.

QD-OLED – Quantum Dot OLED

They are hybrid version of quantum dot and OLED displays that Improves brightness and colour.

Super AMOLEDs

The Samsung's OLED variation is the super AMOLED. Traditionally touch response layer is an additional layer on top of the display. Super AMOLED technology of Samsung integrates the touch response layer and the display into a single layer resulting in reduced power requirement and better visibility in sunlight. An improved version of Super AMOLED is the Dynamic AMOLED with wider range of contrast colours, improved visual comfort through blue light reduction.

Apples trademark names are Retina display, Retina HD display, Liquid Retina Display, Liquid Retina XDR Display, Super Retina HD display, Super Retina XDR display or Retina 4K/5K/6K displays. These apples variations are equivalents of LCDs to super AMOLED displays. Fluid AMOLED is a version of One Plus with high refresh rates and more fluid animations.

Note:

- Nits or Candela per square meter (cd/m^2) is the unit for luminance in international system. It indicates how bright the display is.
- Higher refresh rates indicate higher frame rate; i.e. high rate of image display on screen. It is given in terms of Hz i.e. 90Hz, 120Hz etc.

- The number of pixels (or points) displayed on screen is described by Resolution. A device is given resolution in terms of number of horizontal and vertical pixels. Quarter Video Graphics Array (QVGA): 320x240, High-Definition HD: 720x1280P, Wide Extended Graphics Array WXGA: 1366x768, Full HD: 1080x1920P, Quad HD: 1440x2560P, Ultra HD: 2160x3840P.

Since higher resolution devices use greater number of pixels, require higher data, most of these devices operate using serial communication protocols like USB, I2C and SPI. Only the devices that accept parallel data i.e. the 7-Segment display and 16x2 LCD are presented here.

11.2 Interfacing 7-Segment Display

The most used segment display is the 7-segment display. Seven LEDs represent the seven segments and are arranged in the form of '8' to represent hexadecimal numbers. A representation of the 7-segment LED is presented in Fig 11.1a where the 7 segments are denoted by letters a to g. The LEDs used in the 7-segment display are used in any of the two configurations.

- Common Cathode
- Common Anode

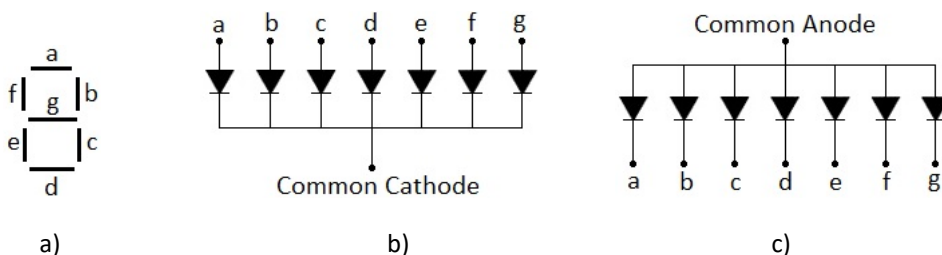


Fig 11.1 Seven Segment LED & configurations
a) Seven Segment LED b) Common Cathode - CC configuration
c) Common Anode – CA configuration

The common cathode and common anode configurations are presented in Fig 11.1b & Fig 11.1c where each LED is represented as a diode. In a common cathode configuration, all the cathodes are connected and require logic 1 to turn ON a specific LED segment. In a common anode configuration, all the anodes are connected and require logic 0 to turn ON a specific LED segment. The set of values (bit combinations) that need to be sent to the 7-segment LEDs (a to g) for displaying a specific hexadecimal number is presented in Table 11.1.

The LED segments that are required for displaying respective characters is presented in Fig 11.2. A typical interfacing of 7-Segment display with PIC18F458 is presented

in Fig 11.3. The 7-Segment is interfaced through current limiting resistors to meet the requirements of 7-Segment display and PIC18 device.

Table 11.1: Numbers & equivalent Values to be sent to port & Seven Segment Display with Common Cathode configuration.

Active High/Common Cathode - CC								No	CC Port Value in Hex	CA = CC Port Value in Hex
PC. 7	PC. 6	PC. 5	PC. 4	PC. 3	PC. 2	PC. 1	PC. 0			
h	g	f	e	d	c	b	a			
0	0	1	1	1	1	1	1	0	3F	C0
0	0	0	0	0	1	1	0	1	06	F9
0	1	0	1	1	0	1	1	2	5B	A4
0	1	0	0	1	1	1	1	3	4F	B0
0	1	1	0	0	1	1	0	4	66	99
0	1	1	0	1	1	0	1	5	6D	92
0	1	1	1	1	1	0	1	6	FD	82/83
0	0	0	0	0	1	1	1	7	07	F8
0	1	1	1	1	1	1	1	8	7F	80
0	1	1	0	1	1	1	1	9	6F	90/98
0	1	1	1	0	1	1	1	A	77	88
0	1	1	1	1	1	0	0	B	7C	83
0	0	1	1	1	0	0	1	C	39	C6
0	1	0	1	1	1	1	0	D	5E	A1
0	1	1	1	1	0	0	1	E	79	86
0	1	1	1	0	0	0	1	F	71	8E

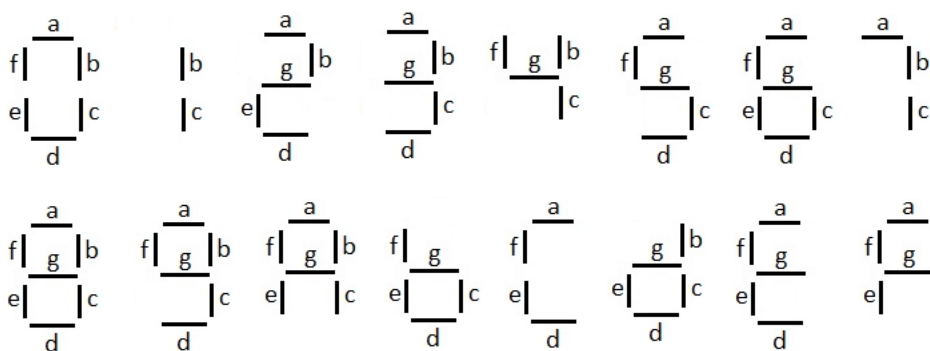


Fig 11.2 LED selection to display different characters

Program 11.1: Read a value from file register 0x10 and display it on 7-segment display.

#INCLUDE <P18F458.INC>; CAN BE OTHER PROCESSOR

	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.
START	ORG	0x100	; Set Origin 0x100 for Main program
	MOVLW	B'00000000'	; WREG = 0x00
	MOVWF	TRISC	; TRISC = 0x00, Make PORTC as Output
L1:	MOVF	0x10,0	; WREG = [0x10], Get the Hex value that need to ; be displayed in 7-Segment.
	CALL	G7SEG	; Call Module G7SEG to get 7-Segment equivalent ; value for a particular hexadecimal number to ; set to PORTC for display
	MOVWF	0x12,0	; [0x12] = WREG, Store 7-Segment equivalent ; value in File Reg 0x12
	MOVWF	PORTC,0	; PORTC = WREG, Send 7-Segment equivalent ; value to 7-Segment Display at PORTC
	CALL	DELAY	; Call Module DELAY to create a delay before ; reading & displaying a new/different hex value.
	BRA	L1	; Branch to L1 to repeat displaying new value in ; 0x12 in 7-Segment display
	ORG	0x200	; Set Origin 0x200 for G7SEG Module
G7SEG	MOVFF	0x10,0x11	; [0x11] = [0x10], Copy File Register content of ; 0x10 to 0x11.
	DECF	0x11,0	; [0x11] = [0x11]-1, Decrement File Reg [0x11].
	INFSNZ	0x11,0	; Increment File Reg [0x11], [0x11] = [0x11] +1 ; and check if not zero, skip next if nonzero. ; To check if the value to be displayed is 0.
	BRA	L2	; If zero branch to L2
	LFSR	0,0x000	; FSR0 = 0x000
L3:	MOVF	POSTINC0,0	; WREG = [FSR0], Get the content of location ; pointed by FSR0. i.e. 7-Seg equivalent value. ; Increment FSR0, FSR0 = FSR0+1
	DECF	0x11, F,0	; [0x11] = [0x11]-1, Decrement File Reg [0x11].
	BNZ	L3	; Branch to L3 if File Reg content [0x11] is not 0
L2:	MOVF	POSTINC0,0	; For loop correction 0 value; WREG = [FSR0], Get ; Content of location pointed by FSR0. i.e. 7-Seg ; Equivalent value. Increment FSR0, FSR0 = FSR0+1
	RETURN		; Return from G7SEG
DELAY	MOVLW	0xE0	; WREG = 0xE0
	MOVWF	0x30	; [0x30] = WREG = 0xE0, delay loop value
	MOVLW	0xF0	; WREG = 0xF0
	MOVWF	0x31	; [0x31] = WREG = 0xF0, delay loop value
LMD1:	DECF	0x30,1	; [0x30] = [0x30]-1, Decrement File Reg [0x30].
	BNZ	LMD1	; Branch to LMD1 if file reg content [0x30] is not 0
LMD2:	DECF	0x31,1	; [0x31] = [0x31]-1, Decrement File Reg [0x31].

BNZ	LMD2	; Branch to LMD2 if Reg content [0x31] is not 0
RETURN		; Return from DELAY
END		

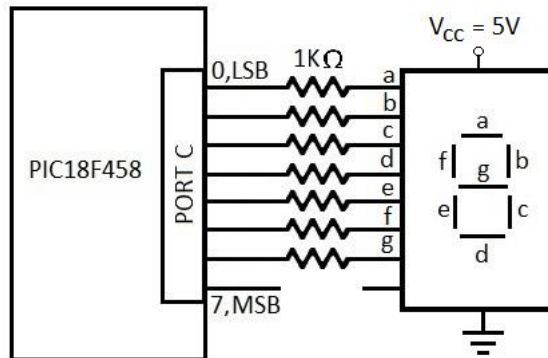


Fig 11.3 Interfacing of 7-Segment LED Display

File Registers ×																
Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
000	C0	F9	A4	B0	99	92	82	F8	80	90	88	83	C6	A1	86	8E
010	06	00	82	00	00	00	00	00	00	00	00	00	00	00	00	00
020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Fig. P11.1: Display of file registers 0x00 to 0x2F.

File register 0x10 entered with a value 0x06, to display 6 on 7-segment display. The equivalent 7-Segment hex value to display 6 is 0x82, stored in 0x12. File registers 0x00 to 0x0F loaded with 7-Seg equivalent hex values for values 0 to F.

Often one seven-segment is not sufficient to display data. So typically, a group of 7-Segments are used to display the given data. A representation that shows the use of three 7-segment's is presented in Fig 11.4. Program 11.3 demonstrates an example.

Program 11.2: Program of 11.1 with 7-Segment equivalent hex values in Flash memory used with table pointer instead of having values stored in File register 0x00 to 0x0F

```

#include<P18F458.INC> ; CAN BE OTHER PROCESSOR
ORG      0x00          ; Set Origin 0x0000 of Flash memory
GOTO     START         ; Go to Start the Main program.
ORG      100H          ; Set Origin 0x100 for Main program
START    MOVLW         B'00000000' ; WREG = 0x00
          MOVWF         TRISC       ; TRISC = 0x00, Make PORTC as Output
          MOVLW         0x00        ; WREG = 0x00
          MOVWF         TBLPTRU,0   ; TBLPTRU = 0x00
          MOVLW         0x10        ; WREG = 0x10
          MOVWF         TBLPTRH,0   ; TBLPTRH = 0x10
          MOVLW         0x00        ; WREG = 0x00
          MOVWF         TBLPTRL,0   ; TBLPTRL = 0x00

```

L1:	MOVF	0x10,0,0	; WREG = [0x10], Get the Hex value that need to ; be Displayed in 7-Segment.
	CALL	G7SEG	; Call Module G7SEG to get 7-Segment equivalent ; value for a particular hexadecimal no to set ; to PORTC for display
	MOVWF	0x12,0	; [0x12] = WREG, Store 7-Segment equivalent ; value in File Reg 0x12
	MOVWF	PORTC,0	; PORTC = WREG, Send 7-Segment equivalent ; value to 7-Segment Display at PORTC
	CALL	DELAY	; Call Module DELAY to create a delay before ; reading & displaying a new/different hex value.
	BRA	L1	; Branch to L2 to repeat displaying new value in ; 0x12 in 7-Segment display
	ORG	0x200	; Set Origin 0x200 for G7SEG Module
G7SEG	MOVFF	0x10,0x11	; [0x11] = [0x10], ; Copy File Register content of 0x10 to 0x11.
	DECF	0x11,0	; [0x11] = [0x11]-1, Decrement File Reg [0x11].
	INFSNZ	0x11,0	; Increment File Reg [0x11], [0x11] = [0x11] +1 ; and check if not zero, skip next if nonzero. ; To check if the value to be displayed is 0.
	BRA	L2	; If zero branch to L2
L3:	TBLRD*+		; TABLAT = [TBLPTR], Get the content of location ; pointed by TBLPTR. i.e. 7-Seg equivalent value. ; Increment TBLPTR, TBLPTR = TBLPTR+1
	DECF	0x11, F,0	; [0x11] = [0x11]-1, Decrement File Reg [0x11].
	BNZ	L3	; Branch to L3 if file reg content [0x11] is not zero
L2:	TBLRD*+		; For loop correction 0 value; TABLAT = [TBLPTR], ; Get the content of the location pointed by ; TBLPTR. i.e. 7-Seg equivalent value. ; Increment TBLPTR, ; TBLPTR = TBLPTR+1
	MOVF	TABLAT,0	; WREG = TABLAT
	RETURN		; Return from G7SEG
DELAY	MOVLW	0xE0	; WREG = 0xE0
	MOVWF	0x30	; [0x30] = WREG = 0xE0, delay loop value
	MOVLW	0xF0	; WREG = 0xF0
	MOVWF	0x31	; [0x31] = WREG = 0xF0, delay loop value
LMD1:	DECF	0x30,1	; [0x30] = [0x30]-1, Decrement File Reg [0x30].
	BNZ	LMD1	; Branch to LMD1 if file reg content [0x30] is not 0
LMD2:	DECF	0x31,1	; [0x31] = [0x31]-1, Decrement File Reg [0x31].
	BNZ	LMD2	; Branch to LMD2 if file reg content [0x31] is not 0
	RETURN		; Return from DELAY
	ORG	1000H	; Set Origin 0x1000 of Table Data in Flash memory
MY_7S	DB	0xC0,0xF9,0xA4,0xB0,0x99,0x92,0x82,0xF8, 0x80,0x90,0x88,0x83,0xC6,0xA1,0x86,0x8E	
	END		

Program 11.3: Display values entered in File Registers 0x20, 0x21 and 0x22 on the three 7-segment displays using table pointer for flash memory.

```

#INCLUDE<P18F458.INC> ; CAN BE OTHER PROCESSOR
ORG      0x00          ; Set Origin 0x0000 of Flash memory
GOTO     START         ; Go to Start the Main program.
ORG      0x100         ; Set Origin 0x100 for Main program
START:   MOVLW          B'00000000' ; WREG = 0x00
        MOVWF          TRISC        ; TRISC = 0x00, Make PORTC as Output
        MOVWF          TRISD        ; TRISD = 0x00, Make PORTD as Output
        MOVLW          0x00         ; WREG = 0x00
        MOVWF          TBLPTRU,0    ; TBLPTRU = 0x00
        MOVLW          0x10         ; WREG = 0x10
        MOVWF          TBLPTRH,0    ; TBLPTRH = 0x10
        MOVLW          0x00         ; WREG = 0x00
        MOVWF          TBLPTRL,0    ; TBLPTRL = 0x00
LM2:     LFSR            0,0x20      ; FSR0 = 0x20
        LFSR            1,0x30      ; FSR1 = 0x30
        MOVLW          0x01         ; WREG = 0x01, 7-Segment Enable start value
        MOVWF          0x12         ; [0x12] = WREG = 0x01
        MOVLW          0x03         ; WREG = 0x03, 7-Segment Count = 3 for loop
        MOVWF          0x13         ; [0x13] = WREG = 0x03
LM1:     MOVF            POSTINC0,0   ; WREG = [FSR0], FSR0 = FSR0 + 1
        MOVWF          0x10,0       ; [0x10] = WREG, Get the value that need to
        ; be Displayed in 7-Segment.
        CALL           G7SEG        ; Call Module G7SEG to get 7-Segment equivalent
        ; value for a particular hexadecimal number to set
        ; to PORTC for display
        MOVWF          POSTINC1,0   ; [FSR1] = WREG, Store 7-Segment equivalent
        ; value in File Reg [FSR1], FSR1 = FSR1 + 1
        MOVWF          PORTC,0      ; PORTC = WREG, Send 7-Segment equivalent
        ; value to 7-Segment Display at PORTC
        RLNCF          0x12,1,0     ; Shift left File reg 0x12 W/O Carry, store in
        ; file reg
        MOVF           0x12,0       ; WREG = [0x12]
        MOVWF          PORTD,0      ; PORTD = WREG, Enable one of the 7-Segment
        ; based on value in File Reg 0x12
        DECF           0x13,1,0     ; [0x13] = [0x13]-1, Decrement File Reg [0x13].
        BNZ            LM1          ; Branch to LM1 if file reg content [0x13] is not 0
        BRA            LM2          ; Branch to LM2 to repeat displaying new value in
        ; 0x20,0x21 & 0x22 in three 7-Segment displays

```

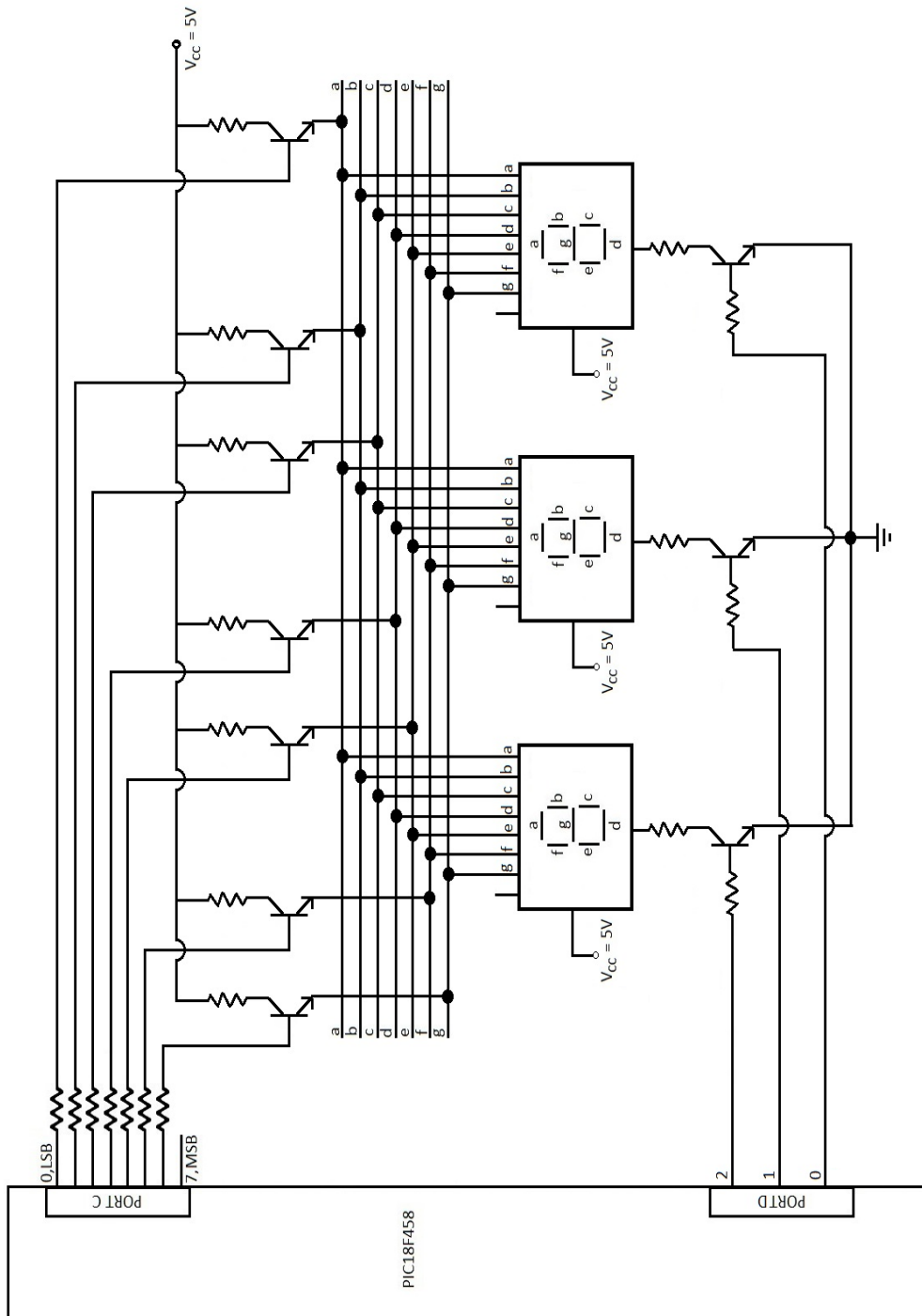


Fig 11.4 Interfacing three 7-Segment Displays

ORG	0x200	; Set Origin 0x200 for G7SEG Module
G7SEG	MOVFF	0x10,0x11 ; [0x11] = [0x10],

			; Copy File Register content of 0x10 to 0x11.
	DECF	0x11,0	; [0x11] = [0x11]-1, Decrement File Reg [0x11].
	INFSNZ	0x11,0	; Increment File Reg [0x11], [0x11] = [0x11] +1
			; and check if not zero, skip next if nonzero.
			; To check if the value to be displayed is 0.
	BRA	L2	; If zero branch to L2
L3:	TBLRD*+		; TABLAT = [TBLPTR], Get the content of location
			; pointed by TBLPTR. i.e. 7-Seg equivalent value.
			; Increment TBLPTR, TBLPTR = TBLPTR+1
	DECF	0x11, F,0	; [0x11] = [0x11]-1, Decrement File Reg [0x11].
	BNZ	L3	; Branch to L3 if file reg content [0x11] is not zero
L2:	TBLRD*+		; For loop correction 0 value; TABLAT = [TBLPTR],
			; Get the content of the location pointed by
			; TBLPTR i.e. 7-Seg equivalent value.
			; Increment TBLPTR, TBLPTR = TBLPTR+1
	MOVF	TABLAT,0	; WREG = TABLAT
	RETURN		; Return from G7SEG
	ORG	1000H	; Set Origin 0x1000 of Table Data in Flash memory
MY_7S	DB	0xC0,0xF9,0xA4,0xB0,0x99,0x92,0x82,0xF8,	
		0x80,0x90,0x88,0x83,0xC6,0xA1,0x86,0x8E	
	END		

Here one 7-segment is activated at a time by controlling the respective transistors that are individually connected to a pin of PORTD.

The logic works as follows.

- Data from file register 0x20 is taken and sent on PORTC, that's received by all the three 7-segments.
- This least significant value is made to display on right/topmost 7-Segment, by enabled its transistor connected to pin 0 of PORTD.
- Data from 0x21 is sent to middle/center 7-Segment, by enabling pin 1 of PORTD.
- Data from 0x22 is sent to left/bottom 7-Segment, by enabling pin 2 of PORTD.

This process is repeated continuously without delay, and the human eye will not be able to detect the disabling of the other 2 two 7-segments as the refreshing rate takes place in the order of microseconds.

11.3 Interfacing LCD Display

This section discusses about Liquid Crystal Display – LCD. The LEDs and 7-Segments do not provide flexibility to display all the characters and symbols. LCD's provide this flexibility as they are made of dot matrix that could be used to display different characters, symbols that were not possible with 7-segment displays. The Fig 11.5 shows the layout of a

7-segment display that contains 16 pins. The pins and their respective functions are presented in Table 11.2

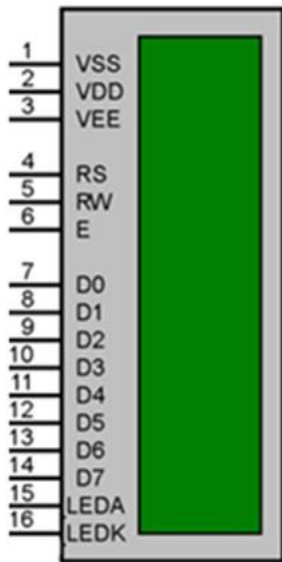


Fig 11.5 LCD Display

Category	Pin No	Pin Name	Function
Power Pins	1	VSS	Ground Pin, Connected to Ground
	2	VDD or Vcc	Voltage Pin +5V
Contrast Pin	3	V0 or VEE	Contrast Setting, connected to Vcc thorough a variable resistor.
Control Pins	4	RS	Register Select, RS=0 Command mode, RS=1 Data mode
	5	RW	Read/ Write pin, RW=0 Write mode, RW=1 Read mode
	6	E	Enable, a high to low pulse need to enable the LCD
Data Pins	7	D0	Data Pins, Stores the Data to be displayed on LCD or the command instructions. For 4-Bit Mode D4-D7 are used.
	.	.	
	14	D7	
Backlight Pins	15	LED+ or A	To power the Backlight +5V
	16	LED- or K	Backlight Ground

Table 11.2: LCD Display Pins

The **VDD** and **VSS** pins are used to supply the power and ground to the LCD. The **LEDA** and **LEDK** are used to supply the power and ground to the backlit light.

The **V0 or VEE** pin is a contrast pin which is connected to the variable terminal of a potentiometer/variable resistor that's tied between Voltage - VCC and Ground - VSS. This provides the variable voltage that is used to change the contrast to display the characters in display.

The **RS** is the Register Select pin that is used to select the mode of the display. This pin is to set at Logic 0 to load a command for configuring the LCD and need to be set at logic 1 to load the data to be displayed on the LCD.

The **RW** is the Read/Write pin that is used to see the status of the LCD or to enable for writing command or data into it. Logic 0 at this pin enables write to LCD and a Logic 1 enables LCD for a read.

The **E** is the Enable pin that operates as read or write enable pin. A read operation is enabled by a rising edge, and a write operation is enabled by a falling edge on this pin.

An LCD internally has a controller which performs the required operations. A write operation is required for writing a command or data to an LCD. A read operation is required for reading the status whether the controller is busy or not. A command or data can only be

written when the controller inside the LCD is ready to accept it. The status, whether the controller is ready or not can be read by accessing the command register.

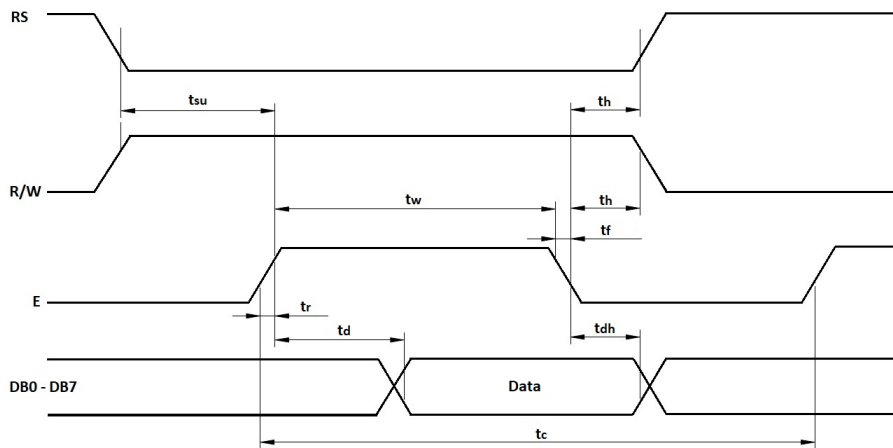


Fig 11.6 a) Timing diagram of Command Register Read operation

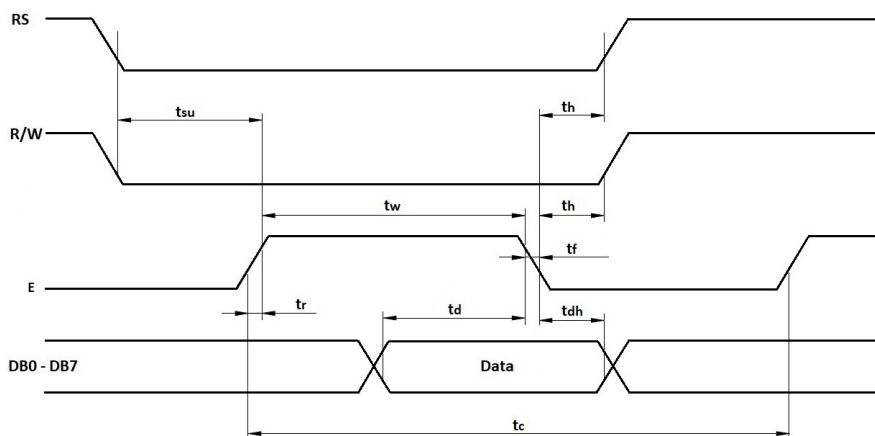


Fig 11.6 b) Timing diagram of Command Register Write operation

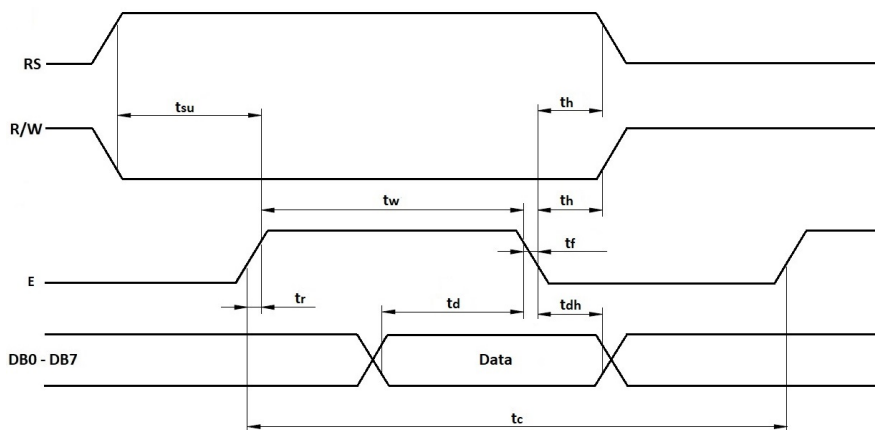


Fig 11.6 c) Timing diagram of Data Write operation

Table 11.3: LCD Parameter Specifications

Pin	Parameter	Description	Write Cycle: Command/ Data, Time ns	Read Cycle: Command, Time ns
Enable	t_c	Enable cycle time	500	500
	t_w	Enable pulse width	300	300
	t_r	Enable rise time	25	25
	t_f	Enable fall time	25	25
RS, RW	t_{su}	RS, R/W Setup time	100	100
	t_h	RS, R/W Address hold time	10	10
DB0-DB7	t_d	Read data output delay	60	60
	t_{dh}	Read data hold time	10	20

A read operation of command register requires RS pin to be set to logic 0, RW pin set to logic 1, a positive edge on E pin. The command register data is presented on the data lines D0-D7 (in 8-bit mode) or D4-D7 (in 4-bit mode). A value of 1 in MSB bit (D7) indicates that the controller is busy and cannot accept command or data. A value of 0 in MSB bit indicates that the LCD is ready to accept. The timing diagram of the Command Read operation is shown in fig 11.6a.

A write operation (command or data to be displayed) can be performed when the controller is ready to accept new information. LCD is configured by writing commands to it. A command write operation is initiated by setting RS pin to logic 0 for selecting command register, RW pin set to logic 0 to set write operation, a negative edge on E pin. The command value may be sent to LCD by placing the data on the data lines D0-D7 (in 8-bit mode) or D4-D7 (in 4-bit mode). The timing diagram of the Command Write operation is shown in fig 11.6b. The data that need to be displayed is also sent to LCD in similar sequence with the exception that RS pin is set to logic 1 for data operation. The timing diagram of the Data Write operation is shown in fig 11.6c. The typical LCD parameter specifications are presented in table 11.3.

The LCD command instruction codes format is presented in table 11.4. A set of most used commands and respective hex code values is given in table 11.5. A typical interfacing of LCD in 8-bit Data mode is presented in fig 11.7a and the 4-bit is presented in fig 11.7b.

COMMAND	COMMAND CODE										COMMAND DESCRIPTION
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
DISPLAY SCREEN CLEAR	0	0	0	0	0	0	0	0	0	1	Clear LCD Display Screen, Set AC/current address to 0 i.e Cursor Reposition to home cursor
CURSOR RETURN HOME	0	0	0	0	0	0	0	0	0	*	Set DDRAM AC/current address to 0 i.e Cursor Reposition to home cursor. Return display to original position. Content unchanged.
INPUT/ENTRY MODE SET	0	0	0	0	0	0	0	0	1	I/D	S Screen action as display character written, Set moving direction of cursor, specify shift of display
DISPLAY ON/OFF SWITCH CONTROL	0	0	0	0	0	0	0	1	D	C	B Set entire display on/off;cursor on/off, blink character on/off
CURSOR/DISPLAY SHIFT	0	0	0	0	0	0	1	S/C	R/L	*	Move cursor and shift whole display, without changing DDRAM contents
FUNCTION SET	0	0	0	0	0	1	DL	N	F	*	Set Data Length, display lines, character font
CGRAM AD SET	0	0	0	0	1	ACG/Character Address					Sets CGRAM Address, send/receive data
DDRAM AD SET	0	0	1	ADD/Display Data Address					Sets DDRAM Address, send/receive data		
READ BUSY AD CT	0	1	BF	AC/Current Address					Read Busy flag if internal operation is being performed, read current address contents		
CGRAM/DDRAM DATA WRITE	1	0	DATA WRITE/Character Type					Write data from CGRAM or DDRAM			
CGRAM/DDRAM DATA READ	1	1	DATA READ/Character Type					Read data from CGRAM or DDRAM			
	I/D=1: Increment/Right; I/D=0: Decrement/Left Mode										
	S=1/0: Shift Screen/Cursor										
	D=1/0, Screen ON/OFF, C=1/0, Cursor ON/OFF, B = 1/0, Cursor Blink/Noblink										
	S/C=1: Display Shift; S/C=0: Cursor Shift										
	R/L=1: Right Shift; R/L=0: Left Shift										
	DL=1: 8D DL=0: 4D Bits per Character										
	N=1: 2R N=0: 1R Rows of Character										
	F=1: 5x10 Style; F=0: 5x7 Style Dots per Character										
	BF=1: Execute Internal Function; Busy										
	BF=0: Command Received, Not Busy										

Table 11.4. LCD Instruction Set

Table 11.5: LCD Commands & their respective Hex Codes

Hex Code	Command to LCD Instruction Register	Hex Code	Command to LCD Instruction Register
01	Clear display screen	02	Return home
04	Decrement cursor (Shift cursor to left)	06	Increment cursor (Shift cursor to right)
08	Display OFF, cursor OFF	0C	Display ON, cursor OFF
0A	Display OFF, cursor ON, blink OFF	0E	Display ON, cursor ON, No blinking
0B	Display OFF, cursor ON, blink ON	0F	Display ON, cursor ON, blinking
05	Shift display right	07	Shift display left
18	Shift entire Display Left	1C	Shift entire Display Right
10	Shift Cursor Left	14	Shift Cursor Right
28	2 lines, 5x7 matrix, 4D per Char	2C	2 lines, 5x10 matrix, 4D per Char
38	2 lines, 5x7 matrix, 8D per Char	3C	2 lines, 5x10 matrix, 8D per Char
80	Force cursor to be beginning of 1st line	C0	Force cursor to be beginning of 2nd line
82	Jump to first line, position 2	C2	Jump to second line, position 2

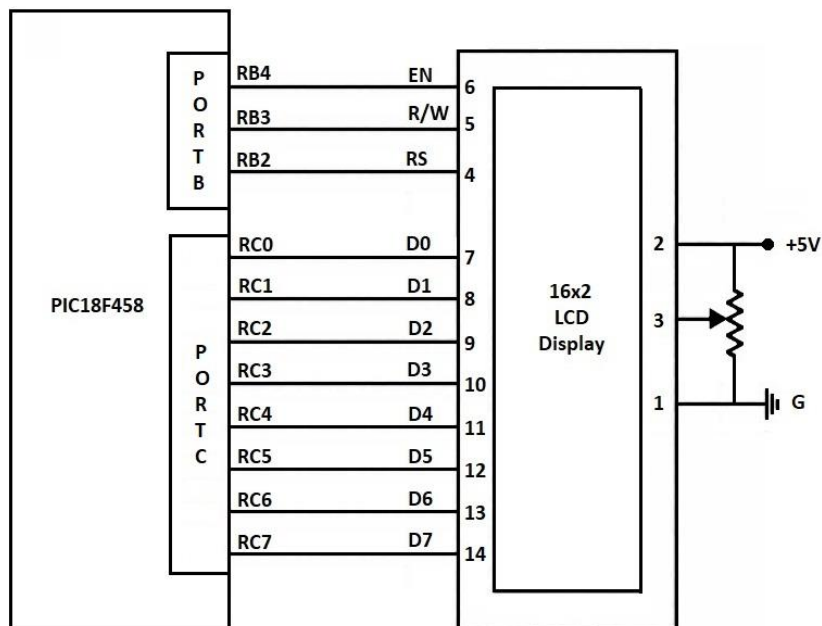


Fig 11.7 a) Interfacing LCD display with 8 Data lines/Bits per Characters

The initialization sequence to configure the LCD and to display a message is presented here along with an equivalent program in program 11.4

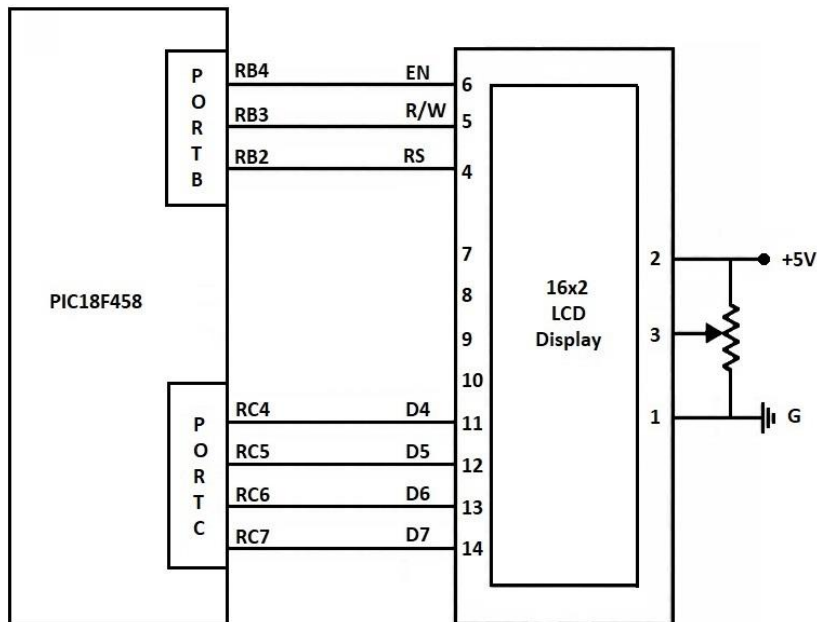


Fig 11.7 b) Interfacing LCD display with 4 Data lines/Bits per Characters
Fig 11.7 Interfacing three 7-Segment Displays

Initialization commands for LCD:

To initialize LCD, the PIC18 must send the following instruction/commands in the following sequence.

Note:

- The checking of LCD controllers Busy Status is done by having RS = 0, RW = 1, EN = +ve edge. This busy checking is performed after every step, and sequence can proceed to the next only if the controller is not busy, ready to accept data.
- The Write operation is given by having RS = 0, RW = 1, EN = -ve edge for each command write to the LCD.

The following data need to be sent to the port connecting the 8-bit data lines of the LCD.

- 0x38 for 8-bit data initialization.
- 0xFH for making LCD ON and initializing the cursor.
- 0x6H for incrementing the cursor, display the characters in the LCD
- 0x1H for clearing the LCD
- 0x80H to start at the first line & character.

This can be followed up with sending of data one-character byte at a time.

Table 11.6: LCD Initialization commands, Hex values, LCD Line characters & their RAM/Data memory addresses.

RAM Address	Command HEX Value	RAM Address	ASCII Value	LCD Line 1 Chars	RAM Address	ASCII Value	LCD Line 2 Chars
0x20	0x38	0x30	0x50	P	0x40	0x4D	M
0x21	0x0F	0x31	0x49	I	0x41	0x69	i
0x22	0x06	0x32	0x43	C	0x42	0x63	c
0x23	0x01	0x33	0x31	1	0x43	0x72	r
0x24	0x80	0x34	0x38	8	0x44	0x6F	o
0x25	0x00	0x35	0x46	F	0x45	0x43	C
0x26	0xC0	0x36	0x78	x	0x46	0x6F	o
		0x37	0x78	x	0x47	0x6E	n
		0x38	0x38	8	0x48	0x74	t
		0x39	0x00	0x00	0x49	0x72	r
					0x4A	0x6F	o
					0x4B	0x6C	l
					0x4C	0x6C	l
					0x4D	0x65	e
					0x4E	0x72	r
					0x4F	0x00	0x00

Program 11.4: Interface an LCD in 8-bit mode and display data (ASCII values) that is stored in RAM data memory. The interfacing is demonstrated in Fig 11.7a.

The program presented here require

Commands from 0x20 RAM address → 38H, 0FH, 06H, 01H, 80H,0

Line 1 data from 0x30 RAM address → "PIC18Fxx8",0

Line 2 data from 0x40 RAM address → "MicroController",0

The stored data is presented in Table 11.6.

Program 11.4a: A simplified version is presented here.

This makes use of number of commands in 0x0DH, number of characters for LCD line 1 in 0x0E and number of characters for LCD line 2 in 0x0F of RAM data memory.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0X00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.
	ORG	0x100	; Set Origin 0x100 for Command Module
CMD	MOVWF	PORTC,0	; LCD Ready, send Command, PORTC =WREG

	BCF	PORTB,2	; RSEL = 0 for Command Selection, PORTB.2 =0
	BCF	PORTB,3	; Set for Write, PORTB.3=0
	BSF	PORTB,4	; Enable Set, PORTB.4=1 for -ve edge, set HIGH
	CALL	DLY1	; Call Delay module to allow required ; pulse width.
	BCF	PORTB,4	; Enable Reset, PORTB.4=0 for -ve edge, ; set LOW
	RETURN		; Return from Command Module
	ORG	0x200	; Set Origin 0x200 for Display Module
DSPY1	MOVWF	PORTC,0	; If LCD is Ready Send Data to LCD, ; PORTC = WREG
	BSF	PORTB,2	; RSEL = 1 for Data Selection, PORTB.2 =1
	BCF	PORTB,3	; Set for Write, PORTB.3=0
	BSF	PORTB,4	; Enable Set, PORTB.4=1 for -ve edge, set HIGH
	CALL	DLY1	; Call Delay module to allow required ; pulse width.
	BCF	PORTB,4	; Enable Reset, PORTB.4=0 for -ve edge, ; set LOW
	RETURN		; Return from Display Module
	ORG	0x350	; Set Origin 0x350 for Delay Module
DLY1	MOVWF	0x00	; [0x00] = WREG, Copy WREG to 0x00
	MOVLW	0xFF	; WREG = 0xFF, No loops for delay
	MOVWF	0x01	; [0x01] = WREG = 0xFF
DL1	DECFSZ	0x01,1,0	; File Reg [0x01] = [0x01]-1. i.e. ; Decrement File Reg
			; 0x01, Skip next instruction if value is 0.
	BRA	DL1	; Branch to DL1 if not 0.
	MOVF	0x00,0	; Restore WREG value from 0x00, ; WREG = [0x00]
	RETURN		; Return from Delay Module
	ORG	0x700	; Set Origin 0x700 for Main program
START	MOVLW	B'00000000'	; WREG = 0x00
	MOVWF	TRISC	; TRISC = 0x00, Make PORTC as Output
	MOVWF	TRISB	; TRISB = 0x00, Make PORTB as Output
	LFSR	1,0x20	; FSR0 = 0x20
L1:	MOVF	POSTINC0,0	; WREG = [FSR0], FSR0 = FSR0+1
	CALL	CMD	; Call Command module.
	CALL	DLY1	; Call Delay module to allow required ; pulse width.
	DECFSZ	0x0D,1	; Reduce the File Reg count, [0x0D] = [0x0D] -1

	BNZ	L1	; If count not 0, Branch to L1
	LFSR	0,0x30	; FSR0 = 0x30
L2:	MOVF	POSTINC0,0	; WREG = [FSR0], FSR0 = FSR0+1
	CALL	DSPY	; Call Display module.
	CALL	DLY1	; Call Delay module to allow required ; pulse width.
	DECF	0x0E,1	; Reduce the File Reg count, [0x0E] = [0x0E] -1
	BNZ	L2	; If count not 0, Branch to L2
	LFSR	0,0x40	; FSR0 = 0x40
L3:	MOVLW	0xC0	; WREG = 0xC0, value for Cursor to 2 nd ; LCD line
	CALL	CMD	; Call Command module, Set Cursor to 2nd ; LCD line
	CALL	DLY1	; Call Delay module to allow required ; pulse width.
	MOVF	POSTINC0,0	; WREG = [FSR0], FSR0 = FSR0+1
	CALL	DSPY	; Call Display module.
	CALL	DLY1	; Call Delay module to allow required ; pulse width.
	DECF	0x0F,1	; Reduce the File Reg count, [0x0F] = [0x0F] -1
	BNZ	L3	; If count not 0, Branch to L3
	END		

Program 11.4b: A generalized version is presented here.

This does not require entering the number of commands, number of characters for LCD line 1 & 2 in 0x0D, 0x0E & 0x0F of RAM data memory. More flexible that can be easily modified for 8-bit and 4-bit modes.

	#INCLUDE<P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.
	ORG	0x100	; Set Origin 0x100 for Command Module
CMD	CALL	DLY1	; Call Delay module between commands to ; allow for LCD to be ready.
	MOVWF	PORTC	; LCD Ready, send Command, PORTC =WREG
	BCF	PORTB,2	; RSEL = 0 for Command Selection, ; PORTB.2 =0
	BCF	PORTB,3	; Set for Write, PORTB.3=0
	BSF	PORTB,4	; Enable Set, PORTB.4=1 for -ve edge, ; set HIGH
	CALL	DLY1	; Call Delay module to allow required ; pulse width.

	BCF	PORTB,4	; Enable Reset, PORTB.4=0 for -ve edge, ; set LOW
	RETURN		; Return from Command Module
	ORG	0x200	; Set Origin 0x200 for Display Module
DSPY	CALL	DLY1	; Call Delay module between commands to ; allow for LCD to be ready.
	MOVWF	PORTC	; If LCD is Ready send Data to LCD, ; PORTC = WREG
	BSF	PORTB,2	; RSEL = 1 for Data Selection, PORTB.2 =1
	BCF	PORTB,3	; Set for Write, PORTB.3=0
	BSF	PORTB,4	; Enable Set, PORTB.4=1 for -ve edge, ; set HIGH
	CALL	DLY1	; Call Delay module to allow required ; pulse width.
	BCF	PORTB,4	; Enable Reset, PORTB.4=0 for -ve edge, ; set LOW
	RETURN		; Return from Display Module
	ORG	0x350	; Set Origin 0x350 for Delay Module
DLY1	MOVWF	0x00	; [0x00] = WREG, Copy WREG to 0x00
	MOVLW	0xFF	; WREG = 0xFF, No loops for delay
	MOVWF	0x01	; [0x01] = WREG = 0xFF
DL1	DECFSZ	0x01,1,0	; File Reg [0x01] = [0x01]-1. i.e. Decrement ; Reg 0x01, Skip next instruction if value is 0.
	BRA	DL1	; Branch to DL1 if not 0.
	MOVF	0x00,0	; Restore WREG value from 0x00, ; WREG = [0x00]
	RETURN		; Return from Delay Module
	ORG	0x600	; Set Origin 0x600 for Send Command or ; Data (Command & Display data) Module
SDCD	NOP		; No Operation
SL2	MOVF	INDF0,0	; WREG = [FSR0]
	TSTFSNZ	POSTINC0,0	; Check content of [FSR0], if it is 0, if it is 0, it ; indicates end of Commands/Data; If 0 go to ; BRA SL1 to RETURN, FSR0 = FSR0 + 1
	BRA	SL1	; Branch to SL1.
	TSTFSNZ	0x05,0	; Check value of 0x05, if it is 0, call Command ; module, if 1 call DSPY module;
	BRA	CL1	; Branch to CL1.
	CALL	DSPY	; Call Display module.
	BRA	SL2	; Branch to SL2. Repeat till Command/Data is ; Complete.
CL1	CALL	CMD	; Call Command module.

	BRA	SL2	; Branch to SL2. Repeat till Command/Data is ; Complete.
SL1	NOP		; No Operation
	RETURN		; Return from SDCD module
	ORG	0x700	; Set Origin 0x700 for Main program
START	MOVLW	B'00000000'	; WREG = 0x00
	MOVWF	TRISC	; TRISC = 0x00, Make PORTC as Output
	MOVWF	TRISB	; TRISB = 0x00, Make PORTB as Output
	LFSR	0,0x20	; FSR0 = 0x20
	BCF	0x05,1,0	; File Reg [0x05] = 0
	CALL	SDCD	; Call SDCD - Send Command/Data module.
	LFSR	0,0x30	; FSR0 = 0x30
	BSF	0x05,1,0	; File Reg [0x05] = 1
	CALL	SDCD	; Call SDCD - Send Command/Data module.
	MOVLW	0xC0	; WREG = 0xC0, value for Cursor to 2 nd ; LCD line
	CALL	CMD	; Call Command module, Set Cursor to 2 nd ; LCD line
	LFSR	0,0x40	; FSR0 = 0x40
	BSF	0x05,1,0	; File Reg [0x05] = 1
	CALL	SDCD	; Call SDCD - Send Command/Data module.
HERE	BRA	HERE	; Branch here.
	END		

Program 11.5: Interface an LCD in 8-bit mode and display data given through program, stored in flash memory. The interfacing is demonstrated in Fig 11.7a.

The program of 11.4 covers data being taken from RAM data memory. It is extended here with data taken from flash program memory. The following program makes use of Flash Memory to read initialization commands, and data to be displayed using Table Pointer. The command values are stored through program from 0x1000, LCD line 1 values from 0x2000 and LCD line 2 from 0x2010 of flash program memory. The data is then transferred to RAM data memory locations starting from 0x20, 0x30 & 0x40 respectively which is then accessed for displaying in LCD. This program also includes checking the busy flag using RDY1 module.

	#INCLUDE<P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.
	ORG	0x100	; Set Origin 0x100 for Command Module
CMD	CALL	RDY1	; Call Ready module to check whether LCD is ; ready to accept command/data
	MOVWF	PORTC	; LCD Ready, send Command, PORTC =WREG

	BCF	PORTB,2	; RSEL = 0 for Command Selection, ; PORTB.2 =0
	BCF	PORTB,3	; Set for Write, PORTB.3=0
	BSF	PORTB,4	; Enable Set, PORTB.4=1 for -ve edge, ; set HIGH
	CALL	DLY1	; Call Delay module to allow required pulse ; width.
	BCF	PORTB,4	; Enable Reset, PORTB.4=0 for -ve edge, ; set LOW
	RETURN		; Return from Command Module
	ORG	0x200	; Set Origin 0x200 for Display Module
DSPY	CALL	RDY1	; Call Ready module to check whether LCD is ; ready to accept command/data
	MOVWF	PORTC	; If LCD is Ready Send Data to LCD, ; PORTC = WREG
	BSF	PORTB,2	; RSEL = 1 for Data Selection, PORTB.2 =1
	BCF	PORTB,3	; Set for Write, PORTB.3=0
	BSF	PORTB,4	; Enable Set, PORTB.4=1 for -ve edge, ; set HIGH
	CALL	DLY1	; Call Delay module to allow required ; pulse width.
	BCF	PORTB,4	; Enable Reset, PORTB.4=0 for -ve edge, ; set LOW
	RETURN		; Return from Display Module
	ORG	0x300	; Set Origin 0x300 for Ready Module
RDY1	BSF	TRISC,7	; Make PORTC.7 as Input to read Busy status ; of LCD. TRISC.7 =1
	BCF	PORTB,2	; RSEL = 0 for Command Selection, ; PORTB.2 =0
	BSF	PORTB,3	; Set for Read, PORTB.3=1
RL1	BCF	PORTB,4	; Enable Reset, PORTB.4=0 for +ve edge, ; set LOW
	CALL	DLY1	; Call Delay module to allow required ; pulse width.
	BSF	PORTB,4	; Enable Set, PORTB.4=1 for +ve edge, ; set HIGH
	CALL	DLY1	; Call Delay module to let LCD respond
	BTFSC	PORTC,7	; Check if PORTC bit 7 is set/reset. Skip next ; instruction if clear & Return, ; else go to BRA inst.
	BRA	RL1	; Branch to RL1
	BCF	TRISC,7	; Make PORTC.7 as output. TRISC.7 =0
	RETURN		; Return from Display Module

	ORG	0x350	; Set Origin 0x350 for Delay Module
DLY1	MOVWF	0x00	; [0x00] = WREG, Copy WREG to 0x00
	MOVLW	0xFF	; WREG = 0xFF, No loops for delay
	MOVWF	0x01	; [0x01] = WREG = 0xFF
DL1	DECFSZ	0x01,1,0	; File Reg [0x01] = [0x01]-1. i.e. Decrement ; Reg 0x01, Skip next instruction if value is 0.
	BRA	DL1	; Branch to DL1 if not 0.
	MOVF	0x00,0	; Restore WREG value from 0x00, ; WREG = [0x00]
	RETURN		; Return from Delay Module
	ORG	0x400	; Set Origin 0x400 for Program memory Read ; & RAM write (Command & Display data) ; Module
P2DM	NOP		; No Operation
PD1	TBLRD*+		; Read Program Memory, TABLAT = ; [TBLPTR], TBLPTR = TBLPTR+1
	MOVF	TABLAT,0	; WREG = TABLAT
	MOVWF	INDF0,0	; [FSR0] = WREG
	TSTFSZ	POSTINC0,0	; Check if [FSR0] content is 0. If 0 skip next ; Instruction & Return.
	BRA	PD1	; Branch to PD1 if [FSR0]! = 0.
	RETURN		; Return from P2DM - Program memory Read ; & RAM write Module
	ORG	0x500	; Set Origin 0x500 for Program memory Read ; & RAM write (Command & Display data) ; Module
PTRS	NOP		; LCD Initialization Command Sequence Start ; Address 0x1000
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TBLPTRU,0	; TBLPTRU = 0x00
	MOVLW	0x10	; WREG = 0x10
	MOVWF	TBLPTRH,0	; TBLPTRH = 0x10
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TBLPTRL,0	; TBLPTRL = 0x00
	LFSR	0,0x20	; FSR0 = 0x20
	CALL	P2DM	; Call P2DM - Program Memory Read & ; RAM Write (Command & Display data) ; Module
	NOP		; LCD Display Message Start Address 0x2000
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TBLPTRU,0	; TBLPTRU = 0x00

	MOVLW	0x20	; WREG = 0x20
	MOVWF	TBLPTRH,0	; TBLPTRH = 0x20
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TBLPTRL,0	; TBLPTRL = 0x00
	LFSR	0,0x30	; FSR0 = 0x30
	CALL	P2DM	; Call P2DM - Program Memory Read & RAM ; Write (Command & Display data) Module
	MOVLW	0x10	; WREG = 0x10
	MOVWF	TBLPTRL,0	; TBLPTRL = 0x10
	LFSR	0,0x40	; FSR0 = 0x40
	CALL	P2DM	; Call P2DM - Program Memory Read & RAM ; Write (Command & Display data) Module
	RETURN		; Return from PTRS - Pointer setting & copy ; for Program Memory Read and RAM write ; Module
	ORG	0x600	; Set Origin 0x600 for Send Command or ; Data (Command & Display data) Module
SDCD	NOP		; No Operation
SL2	MOVF	INDF0,0	; WREG = [FSR0]
	TSTFSNZ	POSTINC0,0	; Check content of [FSR0], if it is 0, if it is 0, it ; indicates end of Commands/Data; If 0 go to ; BRA SL1 to RETURN, FSR0 = FSR0 + 1
	BRA	SL1	; Branch to SL1.
	TSTFSNZ	0x05,0	; Check value of 0x05, if it is 0, call Command ; module, if 1 call DSPY module;
	BRA	CL1	; Branch to CL1.
	CALL	DSPY	; Call Display module.
	BRA	SL2	; Branch to SL2. Repeat till Command/Data is ; Complete.
CL1	CALL	CMD	; Call Command module.
	BRA	SL2	; Branch to SL2. Repeat till Command/Data is ; Complete.
SL1	NOP		; No Operation
	RETURN		; Return from SDCD module
	ORG	0x700	; Set Origin 0x700 for Main program
START	MOVLW	B'00000000'	; WREG = 0x00
	MOVWF	TRISC	; TRISC = 0x00, Make PORTC as Output
	MOVWF	TRISB	; TRISB = 0x00, Make PORTB as Output
	CALL	PTRS	; Copy LCD Initialization Command, Data ; Sequence to RAM
	LFSR	0,0x20	; FSR0 = 0x20

	BCF	0x05,1,0	; File Reg [0x05] = 0
	CALL	SDCD	; Call SDCD - Send Command/Data module.
	LFSR	0,0x30	; FSRO = 0x30
	BSF	0x05,1,0	; File Reg [0x05] = 1
	CALL	SDCD	; Call SDCD - Send Command/Data module.
	MOVLW	0xC0	; WREG = 0xC0
	CALL	CMD	; Call Command module, Set Cursor to 2 nd ; LCD line
	LFSR	0,0x40	; FSRO = 0x40
	BSF	0x05,1,0	; File Reg [0x05] = 1
	CALL	SDCD	; Call SDCD - Send Command/Data module.
HERE	BRA	HERE	; Branch here.
	ORG	1000H	; Point to 0x1000 location of program ; memory. The table data is saved from this ; address.
LCDIN	DB	38H, 0FH, 06H, 01H, 80H,0	; Storing Initializing Sequence values in flash ; program memory
	ORG	2000H	; Point to 0x2000 location of program ; memory. The table data is saved from this ; address.
MYD1	DB	"PIC18Fx8", 0	; Storing LCD Display String "PIC18Fx8" in ; flash program memory
	ORG	2010H	; Point to 0x2010 location of program ; memory. The table data is saved from this ; address.
MYD2	DB	"Microcontr oller",0	; Storing LCD Display String "Microcontroller" ; in flash program memory
	END		

Program 11.6: Interface an LCD in 4-bit mode and display data given through program, stored in flash memory.

Interfacing LCD in 4-bit mode makes use of only 4 data lines D4 to D7 which saves I/O lines. The interfacing is demonstrated in Fig 11.7b. Since it is interfaced in 4-bit mode the value that needs to be sent to LCD must be split/separated nibble wise. This is performed through the SPR1 module presented here. Further the SDCD module and LCDIN variable is modified as given below. The following two modules must be included in Program 11.4 or 11.5 to send data in 4-bit mode. There should be a value of 0x28 instead of 0x38 for the 4-bit mode, and it is shown in initialization of LCDIN variable.

	ORG	0x50	; Set Origin 0x50 for Separator/Split Module
SPR1	MOVWF	0x09,0	; [0x09] = WREG, EX: 0x35

	ANDLW	0xF0,0	; WREG = WREG & 0xF0, EX: 0x30
	MOVWF	0x0A,0	; [0x0A] = WREG, Upper nibble, [0x0A] = 0x30
	MOVF	0x09,0,0	; WREG = [0x09], EX: 0x35
	ANDLW	0x0F,0	; WREG = WREG & 0x0F, EX: 0x05
	SWAPF	WREG,0,0	; Exchange Nibbles, WREG_LD = WREG_HD, ; WREG_HD = WREG_LD, WREG = 0x50
	MOVWF	0x0B,0	; [0x0A] = WREG, Lower nibble, [0x0B] = 0x50
	RETURN		; Return from Command Module
	ORG	0x600	; Set Origin 0x600 for Send Command or Data ; (Command & Display data) Module
SDCD	NOP		; No Operation
SL2	MOVF	INDF0,0	; WREG = [FSR0]
	TSTFSNZ	POSTINC0,0	; Check content of [FSR0], if it is 0, if it is 0, it ; indicates end of Commands/Data; If 0 go to ; BRA SL1 to RETURN, FSR0 = FSR0 + 1
	BRA	SL1	; Branch to SL1.
	CALL	SPR1	; Call Digit Separator Module
	TSTFSNZ	0x05,0	; Check value of 0x05, if it is 0, call Command ; module, if 1 call DSPY module;
	BRA	CL1	; Branch to CL1.
	MOVF	0x0A,0	; Get Upper nibble, WREG = [0x0A]
	CALL	DSPY	; Call Display module.
	MOVF	0x0B,0	; Get Lower nibble, WREG = [0x0B]
	CALL	DSPY	; Call Display module.
	BRA	SL2	; Branch to SL2. Repeat till Command/Data is ; Complete.
CL1	MOVF	0x0A,0	; Get Upper nibble, WREG = [0x0A]
	CALL	CMD	; Call Command module.
	MOVF	0x0B,0	; Get Lower nibble, WREG = [0x0B]
	CALL	CMD	; Call Command module.
	BRA	SL2	; Branch to SL2. Repeat till Command/Data is ; Complete.
SL1	NOP		; No Operation
	RETURN		; Return from SDCD module
	ORG	1000H	; Point to 0x1000 location of program memory. ; The table data is saved from this address.
LCDIN	DB	28H, 0FH, 06H, 01H, 80H,0	; Storing Initializing Sequence values in flash ; program memory

11.4 Interfacing Keypad

This section discusses about Keypad/Keyboards. The keypad or a keyboard is an input device that is a combination of key switches arranged in the form a matrix. A matrix arrangement provides an efficient way to interface the keypad to a microcontroller.

The key switch used in keypads is designed and constructed based on different principles. The common types of switches employed are

- Membrane Switches
- Capacitive Switches
- Hall Effect Switches

The keyboards employ two logic level techniques.

- Active Low Logic
- Active High Logic

The signal levels obtained are presented in Fig 11.7. The interfacing arrangement for the same is provided in Fig 11.8 where each key is represented as a switch. The keyboard matrix has rows (horizontal signal lines) connected to PORTB and columns (vertical signal lines) connected to PORTC.

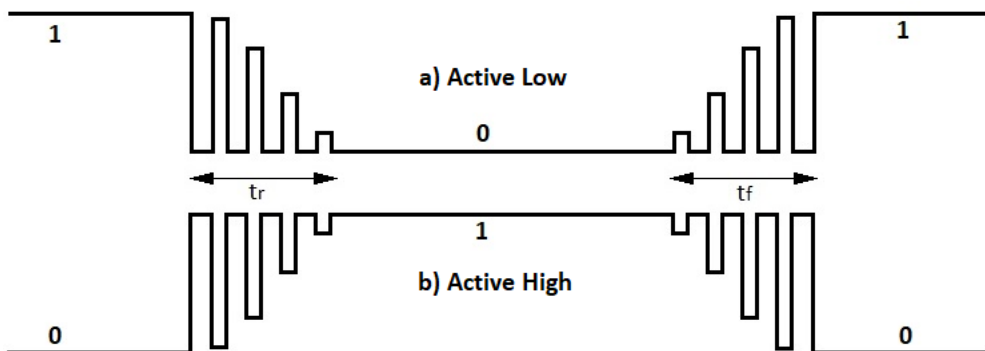


Fig 11.7: Signal levels for pressed switch

Active Low Logic:

The active low logic signalling is shown in Fig 11.7a and the interfacing arrangement for the same is provided in Fig 11.8b. This setup requires logic 0 to be presented at the corresponding row for its activation. A key press in that specific row produces logic 0 at the corresponding intersection column that may be read through the column signal lines connected to PORTC. The column lines remain at logic 1 if no switch is pressed as the lines are powered through V_{cc} of +5V.

Active High Logic:

The active high logic signalling is shown in Fig 11.7b and the interfacing arrangement for the same is provided in Fig 11.8a.

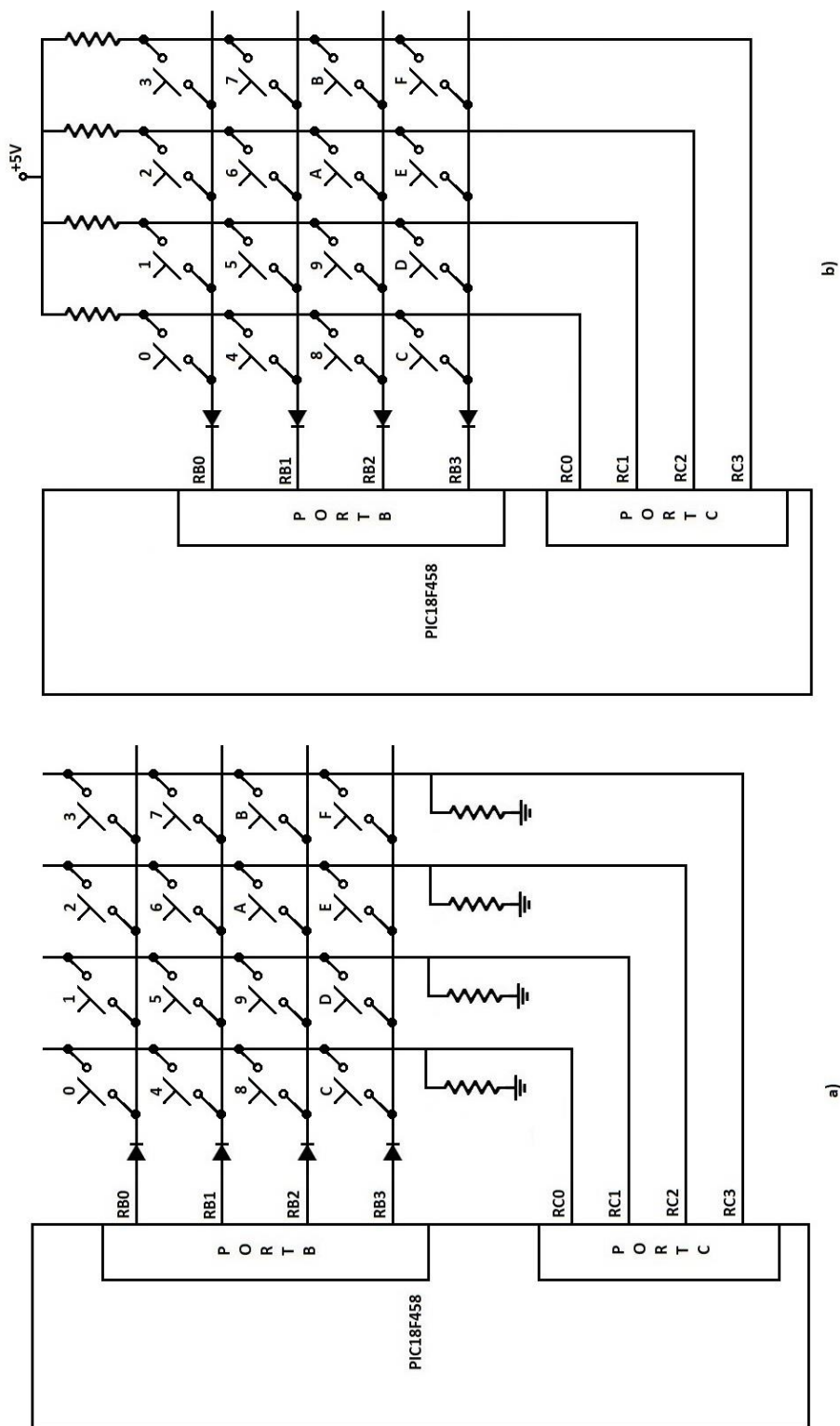


Fig 11.8 Interfacing 4x4 Keypad, a) Active High b) Active Low

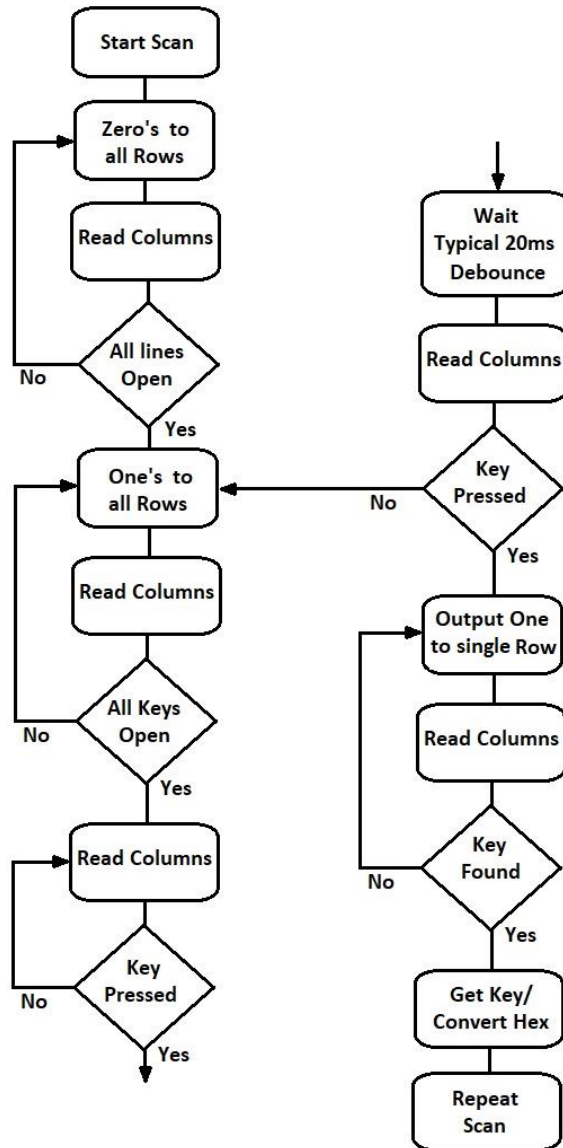


Fig 11.9: Typical Flow chart for Key Detection for Active High logic

This setup requires logic 1 to be presented at the corresponding row for its activation. A key press in that specific row produces logic 1 at the corresponding intersection column that may be read through the column signal lines connected to PORTC. The column lines remain at logic 0 if no switch is pressed as the lines are not powered or are pulled down.

When a key is pressed the key switches oscillates/fluctuates and this is shown as oscillations in Fig 11.7 when changing from logic 0 to logic 1 and vice versa denoted by duration time t_r and t_f . Also, a noise may create an unintentional level change. Hence additional care needs to be taken while determining which key is pressed. A flow chart

provides a typically followed procedure. It is presented in Fig 11.9. For simplicity the following program only presents the required logic for key detection and does not present the code for keys open, debounce etc.

Table 11.7: Key & required Starting Values, PORTB Row activation values & PORTC Column Read values for Active High & Low Logic

Key Pressed	File Reg 0x20 Starting Values	Active High		Active Low	
		PORTB Row Activation Values	PORTC Column Read Values	PORTB Row Activation Values	PORTC Column Read Values
0x00	0xFF	0x01	0x01	0xFE	0xFE
0x01			0x02		0xFD
0x02			0x04		0xFB
0x03			0x08		0xF7
0x04	0x03	0x02	0x01	0xFD	0xFE
0x05			0x02		0xFD
0x06			0x04		0xFB
0x07			0x08		0xF7
0x08	0x07	0x04	0x01	0xFB	0xFE
0x09			0x02		0xFD
0x0A			0x04		0xFB
0x0B			0x08		0xF7
0x0C	0x0B	0x08	0x01	0xF7	0xFE
0x0D			0x02		0xFD
0x0E			0x04		0xFB
0x0F			0x08		0xF7

Program 11.7: Interface a 4x4 keypad using active high logic and store the entered keys in RAM data memory. The interfacing arrangement is shown in Fig 11.8a. The appropriate values required are presented in Table 11.7.

	#INCLUDE<P18F458.INC>; CAN BE OTHER PROCESSOR		
	ORG	0X00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.
	ORG	0x100	; Set Origin 0x100 for Delay Module
DLY1	MOVWF	0x00	; [0x00] = WREG, Copy WREG to 0x00
	MOVLW	0xFF	; WREG = 0xFF, No loops for delay
	MOVWF	0x01	; [0x01] = WREG = 0xFF
DL1:	DECFSZ	0x01,1,0	; File Reg [0x01] = [0x01]-1. ; i.e. Decrement File Reg ; 0x01, Skip next instruction if value is 0.

	BRA	DL1	; Branch to DL1 if not 0.
	MOVF	0x00,0	; Restore WREG value from 0x00, ; WREG = [0x00]
	RETURN		; Return from Delay Module
	ORG	0x200	; Set Origin 0x200 for Conversion Module
CNV1	MOVLW	0x04	; WREG = 0x04
	MOVWF	0x10	; [0x10] = WREG = 0x04
	MOVF	PORTC,0	; WREG = PORTC, Read Column Values
LM1:	INCF	0x20,1,0	; [0x20] = [0x20] +1, Increment File Reg ; contents, store in same File Reg
	RLCF	WREG,0	; Rotate WREG left through Carry
	BC	LM2	; If Carry, i.e. Key pressed got to LM2 ; BNC for active Low logic with appropriate ; active low logic values as per Table 11.7
	DECF	0x10,1	; [0x10] = [0x10] - 1, Decrement File Reg [0x10] ; content
	BNZ	LM1	; If not zero branch to LM1
	BRA	LM3	; If zero branch to LM3 & Return
LM2:	MOVF	0x20,0	; WREG = [0x20]
	MOVWF	POSTINC0,0	; Copy pressed key to [FSR0], RAM Data ; memory location
LM3:	RETURN		; Return from CNV1 module
	ORG	0x500	; Set Origin 0x500 for Main Program
START	MOVLW	B'00000000'	; WREG = 0x00
	MOVWF	TRISB	; TRISB = 0x00, Make PORTB as Output
	MOVLW	B'11111111'	; WREG = 0xFF
	MOVWF	TRISC	; TRISC = 0xFF, Make PORTC as Input
	LFSR	0, 0x30	; FSR0 = 0x30
L1:	MOVLW	0x01	; WREG = 0x01, 1st Row Activation value
	MOVWF	PORTB,0	; Activate 1st Row, PORTB = 0x01
	MOVLW	0xFF	; WREG = 0xFF, Initial Key Value for 1st Row
	MOVWF	0x20,0	; [0x20] = 0xFF, Store Initial Key Value for ; 1st Row
	CALL	CNV1	; Call Conversion - CNV1 module ; i.e. Get Key Value
	MOVLW	0x02	; WREG = 0x02, 2nd Row Activation value
	MOVWF	PORTB,0	; Activate 2nd Row, PORTB = 0x02
	MOVLW	0x03	; WREG = 0x03, Initial Key Value for 2nd Row
	MOVWF	0x20,0	; [0x20] = 0x03, Store Initial Key Value for ; 2ndRow
	CALL	CNV1	; Call Conversion - CNV1 module ; i.e. Get Key Value

	MOVLW	0x04	; WREG = 0x04, 3rd Row Activation value
	MOVWF	PORTB,0	; Activate 3rd Row, PORTB = 0x04
	MOVLW	0x07	; WREG = 0x07, Initial Key Value for 3rd Row
	MOVWF	0x20,0	; [0x20] = 0x07, Store Initial Key Value for ; 3rd Row
	CALL	CNV1	; Call Conversion - CNV1 module ; i.e. Get Key Value
	MOVLW	0x08	; WREG = 0x08, 4rth Row Activation value
	MOVWF	PORTB,0	; Activate 4rth Row, PORTB = 0x08
	MOVLW	0x0B	; WREG = 0x0B, Initial Key Value for 4rth Row
	MOVWF	0x20,0	; [0x20] = 0x0B, Store Initial Key Value for ; 4rthRow
	CALL	CNV1	; Call Conversion - CNV1 module ; i.e. Get Key Value
	BRA	L1	; Branch to L1
	END		